Approximating Memorization Using Loss Surface Geometry for Dataset Pruning and Summarization

Andrea Agiollo*†
University of Bologna
Bologna, Italy
andrea.agiollo@unibo.it

Young In Kim*
Purdue University
West Lafayette, IN, USA
kim3531@purdue.edu

Rajiv Khanna
Purdue University
West Lafayette, IN, USA
rajivak@purdue.edu

Abstract

The sustainable training of modern neural network models represents an open challenge. Several existing methods approach this issue by identifying a subset of *relevant* data samples from the full training data to be used in model optimization with the goal of matching the performance of the full data training with that of the subset data training. Our work explores using *memorization* scores to find representative and atypical samples. We demonstrate that memorization-aware dataset summarization improves the subset construction performance. However, computing memorization scores is notably resource-intensive. To this end, we propose a novel method that leverages the discrepancy between sharpness-aware minimization and stochastic gradient descent to capture data points atypicality. We evaluate our metric over several efficient approximation functions for memorization scores – namely *proxies* –, empirically showing superior correlation and effectiveness. We explore the causes behind our approximation quality, highlighting how typical data points trigger a flatter loss landscape compared to atypical ones. Extensive experiments confirm the effectiveness of our proxy for dataset pruning and summarization tasks, surpassing state-of-the-art approaches both on canonical setups – where atypical data points benefit performance – and few-shot learning scenarios—where atypical data points can be detrimental.

CCS Concepts

• **Computing methodologies** → **Neural networks**; *Image representations*; • **Theory of computation** → *Sample complexity and generalization bounds*; *Models of learning*.

Keywords

Neural Networks, Data-efficient Learning, Memorization, Flatness

ACM Reference Format:
Andrea Agiollo, Young In Kim, and Rajiv Khanna. 2024. Approximating Memorization Using Loss Surface Geometry for Dataset Pruning and Summarization. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '24), August 25–29, 2024, Barcelona, Spain.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3637528.3671985

*Both authors contributed equally.
†Work done while the author was an intern at Purdue University.

This work is licensed under a Creative Commons Attribution International 4.0 License.

KDD '24, August 25–29, 2024, Barcelona, Spain
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0490-1/24/08
https://doi.org/10.1145/3637528.3671985

1 Introduction

Machine and Deep Learning (ML & DL) applications are becoming ubiquitous in modern society. Their growth in popularity is mainly due to the superhuman performance reached by Neural Networks (NNs) on several learning tasks, such as object detection [11, 45], sentiment analysis [3, 44] and many more [2, 39]. However, the optimization of such approaches for specific learning tasks necessitates substantial amounts of data and computational resources. Moreover, we currently lack a comprehensive understanding of how NNs learn from complex datasets and what constitutes a *good* or *clean* learning example [6, 9]. Consequently, research efforts have focused on *data efficiency* – aiming at learning effective models with fewer data – and *sample importance*—measuring the impact of each data sample on NN optimization.

Our focus in this work is *dataset pruning and summarization* [26, 36, 38, 48, 51] aiming at identifying a smaller subset of training data carrying highly informative samples to be used for possible downstream tasks. An example of this could be huge molecule datasets [47] in chemistry, where drug discovery hinges upon searching through over 10^{60} molecules. But compressing the data by selecting fewer molecules that are predictive of a certain property can accelerate the drug discovery task. There are similar problems in metagenomics [35], high energy physics [1], and neuroscience [41].

Within this context, various techniques propose to identify relevant samples of the dataset – either statically before the training starts [38] or dynamically as the training proceeds [26] – and optimize the NN model using solely those samples. Concurrently, studies have delved into the complexity characteristics of data samples during NN training, examining aspects such as forgetting [49] and memorization [13]. Recent empirical findings suggest that there exists a profound relationship between sample complexity – also referred to as typicality [49] or cleanliness [16] – and their relevance for data efficiency. Building on these insights, we propose leveraging memorization scores – as originally defined in Feldman and Zhang [13] – to boost data summarization performance. Our experimental results benchmarked against several state-of-the-art approaches highlight the superiority of our approach for selecting both small and large informative subsets of data. Notably, we observe that low memorization samples prove to be ineffective for large subsets summarization, while they are the priority whenever small subsets are considered. This phenomenon is explored from the perspective of Neural Tangent Kernel (NTK) [15] dynamics, highlighting a correlation between sample memorization and the NTK velocity.

While memorization scores prove effective for data summarization, their extraction requires significant computational resources, limiting scalability across different datasets and learning setups. For

example, to estimate memorization scores for individual data points of the CIFAR100 dataset, Feldman and Zhang [13] trained 4000 NN models. To alleviate this problem, we direct our attention to more viable proxies of memorization scores. Specifically, we leverage the relationship between the loss landscape of NNs and memorization scores. Several empirical studies have demonstrated superior generalization performance of wider minima [14, 22] which can be sought using optimization algorithms such as Sharpness-Aware Minimization (SAM). A recent study demonstrated that this improved performance can be largely attributed to increased performance on data samples of higher memorization scores, while on the data points of lower memorization scores the two algorithms (SGD and SAM) perform similarly [29]. In addition, we hypothesize and observe that data samples with lower memorization scores trigger a flatter loss landscape, while SAM induces a larger flattening effect on higher memorization samples (see Figure 3 for discussion). We reverse engineer this insight and propose a novel method, namely SAM − SGD (SAMIS), that leverages the discrepancy between SAM and Stochastic Gradient Descent (SGD) on individual data points to effectively approximate memorization scores. In addition to SAMIS, we also propose additional faster approximation *proxies* and empirically show their correlation with actual memorization values, along with their improvements over other available metrics. Finally, we test if – and to what extent – the correlation between memorization and its proxies propagate to data-efficient learning, thoroughly testing the performance of our memorization-aware approaches over three datasets and against eight existing and state-of-the-art approaches.

We summarize our main findings as follows: *(i)* memorization scores can be used for data summarization effectively – identifying a new state-of-the-art; *(ii)* NTK velocity is proportional to memorization scores of data samples and it can be used to identify strong data summarization subsets; *(iii)* clean typical samples – characterized by low memorization – trigger flatter loss minima compared to atypical (rare or noisy) – characterized by high memorization – samples; *(iv)* leveraging the discrepancy between SAM and SGD can effectively approximate memorization scores; *(v)* high fidelity memorization proxies attain similar performance of memorization scores for data summarization, surpassing state-of-the-art solutions while also being scalable.

2 Related Work

This work lies at the intersection between the broad areas of measuring data sample importance, dataset summarization, and dataset distillation approaches. Accordingly, we here summarize the most relevant works in these fields.

Sample importance. The concept of sample importance permeates various areas of research in ML, as it allows for example to speed up training [23, 24] and reduce annotation costs [42]. Measuring sample importance represents a complex task by design, as it depends on many factors characterizing the learning process of ML models. Therefore, several approaches consider studying the importance of data samples through the lens of various properties of ML models and learning algorithms. Feldman and Zhang [13] analyze the sample memorization issue occurring in NNs, defining memorization and influence metric and showing how typical, clean samples are characterised by low levels of memorization. Similarly, Toneva et al. [49] empirically identify a correlation between sample complexity and the corresponding forgetting events at training time. Garg and Roy [16] tackle the sample importance problem through the lens of loss curvature, showing how the loss curvature relates to the cleanliness of the data points, with low curvature samples corresponding to clean, typical samples.

Dataset summarization. This task refers to the problem of selecting the most relevant subset of data samples from a large dataset. The most popular approaches in this context rely on the contribution of a sample to the loss or its gradient. For example, both Grad-Match [25] and CRAIG [36] select the weighted subsets whose gradients best approximate the loss gradient on the entire training dataset every few epochs during training. CRAIG converts the gradient-distance optimization problem into a submodular function solvable using a greedy approach, while GradMatch leverages an orthogonal matching pursuit algorithm to closely match the gradient of the loss function on both training and validation samples. Similarly, InfoBatch [40] uses soft pruning based on loss value to achieve higher efficiency. While effective, such approaches require frequently re-selecting the data subset during training, increasing the computational overhead. To overcome this issue GraNd [38] proposes to approximate the L2 norm of the gradient early on in training. Few other approaches consider selecting data subsets based on the data decision boundary – relying on the assumption that the points closest to the boundary are the most informative – and differ depending on the distance measure they use [10, 34]. Similarly, cluster-based dataset summarization approaches consider identifying relevant samples based on how well they cluster together and differ depending on the clustering approach or metric used [5, 19]. Finally, some recent approaches consider the minimization of the data summarization loss directly as a bilevel optimization problem, either maximizing the log-likelihood on a held-out validation set of samples [26], minimizing directly the loss function [27], or leveraging submodular optimization [20, 21].

Dataset distillation. Dataset distillation removes the restriction of uneditable elements characterizing dataset summarization, aiming at distilling the knowledge of the large original dataset into a small synthetic set [31]. In this context, gradient matching approaches represent popular solutions, aiming at imitating the training achieved by a model on the given dataset matching the gradients induced by the synthetically generated samples and the original ones [28, 53]. Similar works propose to directly match the long-range training trajectory between the target dataset and the synthetic dataset, training models on the target dataset and collecting the expert training trajectory into a buffer [4, 8]. Although effective, dataset distillation approaches introduce an added layer of obscurity to the NN training process, as the constructed optimal condensed synthetic data lacks interpretability in terms of attribution of downstream performance to real-world entities. For example, Wang et al. [50] distill 60K images of MNIST into only 10 images while achieving similar test accuracy. The distilled synthetic images, however, are by no means interpretable to the human eye for further analysis. In this paper, we focus solely on the dataset summarization task, though exploring the extension of our ideas to distillation settings could be an interesting future direction.

3 Memorization for Data Summarization

3.1 Problem Definition

Consider a learning task in which the given training set is defined as $\mathcal{T} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{|\mathcal{T}|}$, where $\mathbf{x}_i \in \mathcal{X}$ is the input data sample, $y_i \in \mathcal{Y}$ is the ground-truth label of sample $\mathbf{x}_i$, and $\mathcal{X}$ and $\mathcal{Y}$ denote the input and output spaces, respectively. $\mathcal{V} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{|\mathcal{V}|}$ defines a held-out validation set of samples. Given a neural network model h with parameters θ and a loss function $\mathcal{L}$, we can define the loss on a set S of instances as $\mathcal{L}(\theta, \mathcal{S}) = \sum_{i \in \mathcal{S}} \mathcal{L}(\theta, \mathbf{x}_i, y_i)$. Similarly, $\mathcal{L}(\theta, \mathcal{T})$ denotes the training loss over the full training set $\mathcal{T}$, and $\mathcal{L}(\theta, \mathcal{V})$ the corresponding validation loss. Under these conditions, the data summarization task aims at identifying the most informative subset $\mathcal{S} \subset \mathcal{T}$ such that $|\mathcal{S}| < |\mathcal{T}|$ – preferably $|\mathcal{S}| << |\mathcal{T}|$ – so that the model $h(\theta_{\mathcal{S}})$ trained on $\mathcal{S}$ achieves similar generalization performance to the model $h(\theta_{\mathcal{T}})$ trained on the whole training set $\mathcal{T}$. Mathematically, we can formulate the data summarization problem by searching for the subset $\mathcal{S}$ that allows to minimize the validation loss of the model optimized on the same subset of training samples $\mathcal{S}$:

$$\underset{\mathcal{S} \subset \mathcal{T}}{\operatorname{argmin}} \left\{ \mathcal{L}\left(\underset{\theta}{\operatorname{argmin}} \left\{ \mathcal{L}(\theta, \mathcal{S}) \right\}, \mathcal{V} \right) \right\}. \tag{1}$$

Throughout this manuscript, we refer to the data subset size by the ratio of training data samples selected, namely $\mathcal{P} = |\mathcal{S}|/|\mathcal{T}|$.

3.2 Memorization Definition

Feldman and Zhang [13] define memorization to quantify the self-importance of a data point when predicting on itself, rather than from other training data points. Formally:

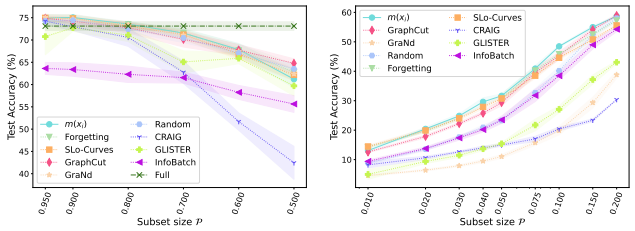
$$m(\mathbf{x}_i) = P\left[h^{\mathcal{A}}_{\mathcal{T}}(\mathbf{x}_i) = y_i \right] - P\left[h^{\mathcal{A}}_{\mathcal{T}^{\setminus i}}(\mathbf{x}_i) = y_i \right], \tag{2}$$

where $\mathcal{A}$ represents a training algorithm optimizing the parameters θ of model h on the training dataset $\mathcal{T}$ and $\mathcal{T}^{\setminus i}$ refers to the dataset $\mathcal{T}$ with $(\mathbf{x}_i, y_i)$ removed. Intuitively, a data point (x_i, y_i) is being memorized (rather than learnt from rest of the data) if its prediction on $\mathbf{x}_i$ changes significantly once $(\mathbf{x}_i, y_i)$ is added to the dataset.

3.3 Experiments

Several works show how data pruning is effective whenever easy to learn samples are removed from the training set if the selected subset size $\mathcal{P}$ is sufficiently large [38, 49]. Conversely, Garg and Roy [16] highlight how relying on clean samples achieves a relevant level of performance over few-shot learning setups in data summarization—i.e., small values of $\mathcal{P}$. Concurrently, the empirical investigations in [13] show how memorization scores are useful for identifying and separating clean – i.e., low memorization – and complex – i.e., high memorization – samples. Accordingly, we here propose to leverage memorization scores of training samples for data summarization.

We analyse the memorization-aware data summarization performance on the two extremities of the $\mathcal{P}$ spectrum. Accordingly, we define a *few-shot data summarization* setup with $\mathcal{P} = [0.01, 0.2]$ and a *data pruning* setup with $\mathcal{P} = [0.5, 0.95]$. We use memorization estimate $m_i = m(\mathbf{x}_i)$ as the selection metric, choosing samples with the highest (lowest) values to construct our subset in the data pruning (few-shot) setup respectively. To illustrate the effectiveness of memorization scores for data summarization, we first select the points with the lowest/highest m_i and then train new randomly initialized networks on these points using the standard cross entropy loss. For more details about the experiment setup see Appendix D[1].



(a) Data pruning **(b) Few-shot data summarization**

Figure 1: Memorization-aware data summarization against state-of-the-art on CIFAR100. Memorization outperforms the baselines over most $\mathcal{P}$.

We compare memorization-aware data summarization against 8 baseline approaches: *(i)* random uniform sampling, *(ii)* Glister [26], *(iii)* CRAIG [36], *(iv)* GraphCut [21], *(v)* GraNd [38], *(vi)* Forgetting [49], *(vii)* SLo-Curve [16], and *(viii)* InfoBatch [40]. For methods that require training before subset selection, we optimize the model for 100 epochs before selecting the subset. The implementation of the baselines follows the one available in the DeepCore library [18]. We select the CIFAR100 dataset [30] as the training target, since its memorization scores are made readily available by Feldman and Zhang [13][2].

Figure 1 shows the achieved test accuracy on the CIFAR100 dataset for data summarization using our memorization-aware approach against the state-of-the-art. Data summarization relying on memorization scores achieves performance improvement over all selected baselines through most values of $\mathcal{P}$, both in the data pruning and few-shot setups, reaching up to 5% higher accuracy against all baselines for $\mathcal{P} = 0.1$. Interestingly, the proposed sampling approach largely outperforms several popular approaches such as CRAIG and GLISTER over the whole spectrum of $\mathcal{P}$, reaching almost 2× performance on the few-shot setup. These results highlight how sampling based on memorization represents the best approach for data pruning, corroborating the *sample cleanliness* hypothesis. Armed with this insight, we further investigate if – and to what extent – it is possible to empirically approximate memorization, and how such proxy metrics behave for data pruning.

3.4 Why are Memorization Scores effective?

To provide an insight on the reason why selecting either high or low memorization samples is a good idea, we consider analysing the evolutionary dynamics of the NTK over the memorization scores. It is well-understood that the data-dependent NTK – defined as the Gram matrix of the logit Jacobian – evolves with high velocity for unlearnt samples, while its velocity stabilizes to smaller values whenever samples are learnt. Therefore, we analyse the

[1]Source code is publicly available at https://github.com/AndAgio/SAMIS
[2]https://pluskid.github.io/influence-memorization/

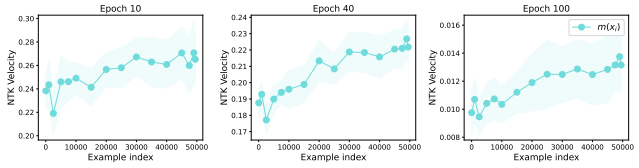


Figure 2: NTK velocity for contiguous subsets of images when ResNet18 is trained on CIFAR100. Examples are sorted in ascending order by memorization scores. 50 contiguous images are selected to compute each point in the plot. Velocity is directly proportional to memorization across all epochs.

contribution of memorized samples to the NTK gram matrix evolution, empirically evaluating the evolution as the velocity of a NTK submatrix over a subset of images in a scale-invariant way, following Fort et al. [15].

Figure 2 shows the NTK velocity – defined as the cosine distance between two NTK gram matrices – on the given subset of contiguous memorization scores, one computed at epoch t, and another one at epoch $t+1$. The NTK velocity is proportional to the samples' memorization scores. This implies that slow samples are more effective whenever few-shot learning is considered, while high-velocity samples are best for constructing large subsets. We hypothesize that when the selected data subset is large enough – i.e., data pruning setup –, it is possible to disregard the samples having velocity close to zero as they represent easy-to-learn data carrying redundant information. Indeed, for sufficiently large sampling from the high memorization tail, we gather a very informative mixture of hard-to-learn and not-so-easy-to-learn examples. Meanwhile, whenever the selected subset is very small – i.e., few-shot setup –, focusing on high memorization samples would be detrimental – as their high velocity even in later epochs implies hard-to-learn samples that have higher memorization affinity and will not generalize well.

Interestingly, it is also possible to notice a small – yet relevant – drop of velocity for the very last samples having memorization scores equal to 1. We hypothesize that the kernel velocity drops off as these examples are usually very unrepresentative – e.g., singleton outliers or mislabeled examples – and are the most difficult for the model to learn, even till later epochs of the training process.

4 Memorization Proxy Using SAMIS

As observed in Section 3, memorization scores are effective in data summarization. However, according to Feldman and Zhang [13], memorization is computationally hard because estimating memorization scores with a standard deviation of σ requires running the training algorithm on the order of $1/\sigma^2$ times for every data instance. Despite the effort to reduce the computational complexity by leveraging an estimator of the expected memorization on a random subset of $\mathcal{T}$, the memorization computation on the CIFAR100 dataset still requires training 4000 different NNs for a reasonable estimate [13]. Therefore, it is necessary to identify possible approximations for the memorization scores that are less resource-demanding. To this end, we introduce a novel method for approximating memorization scores leveraging the discrepancy between SAM and SGD. We illustrate the motivation behind our method and compare against four other different potential proxies, two of which we have developed.

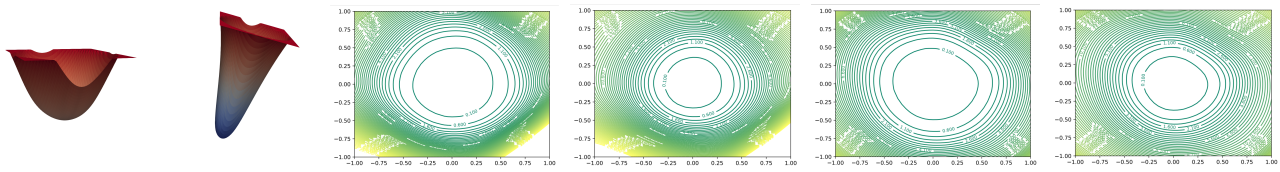
4.1 Typical Samples Trigger Flatter Minima

With the goal of looking for good memorization proxies, we explore the loss surface flatness through the lens of low and high memorization samples. While the loss landscape of a large NN may be highly non-convex and highly intricate, there is increasing evidence of flatter/wider minima translating to better generalization performance [14, 22]. In a recent study, Kim et al. [29] observe that the difference in generalization performance between wider and sharper minima is largely due to their corresponding good vs bad performance respectively on high-memorized samples. Further, it is also known atypical (i.e. high-memorized) training samples from rare subclasses are beneficial in accurately classifying test samples from the same rare subclasses [12].

Taking a step further, *we conjecture that low-memorized samples induce flatter minima inherently when trained with SGD compared to high-memorized samples.* For empirical validation, we train two models with SGD using different subsets of the CIFAR100 dataset and visualize the loss surface following Li et al. [32]. The first model is trained using the fraction of low-memorization examples while the second is trained on the same fraction of high-memorization samples. We choose 60% as an adequate fraction resulting in similar test accuracy across the two models. Figures 3a and 3b present the loss landscape visualization of both models evaluated using the full training set. The results corroborate our hypothesis, as the model trained with low-memorization samples finds a flatter minimum compared to the high-memorization model. Importantly, these results generalize across architectures and datasets as shown in Appendix B.

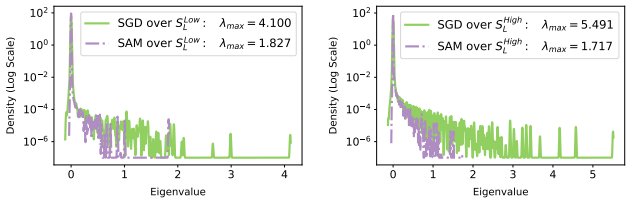
SAM is an optimization algorithm designed to reach a flatter optimum by minimizing the maximum loss in the vicinity of the optimum [14]. We study the effect of this optimization method on the loss surface of data points at either end of the memorization spectrum compared to traditional SGD. We hypothesize that the difference in flatness between the two optimizers is exacerbated for high memorized samples, as the above experiment shows that the minimum obtained using SGD is already quite flat for low memorization samples. On the other hand, we hypothesize that for high memorization samples SAM would have a stronger flattening effect compared to SGD. If such a discrepancy can be established, we can leverage this contrast to approximate the memorization level of all the data points. Finally, we want to assert if the conclusion from the above experiment can be extended to models trained on the full dataset and not just on the subset of training samples. We report our findings in Figure 3. We train two models on the full training dataset using SGD and SAM, and, we compute the loss around the minima for samples at the low and high end of the memorization spectrum over both models.

As Figures 3c to 3f show, low memorized samples trigger flat minima for both optimizers, confirming the findings of Figures 3a and 3b, and being consistent with [14]. However, the difference in flatness between SAM and SGD models is greater for high memorization samples. To reassert this finding, we also compute the



(a) Low mem (b) High mem (c) SGD - Low mem (d) SGD - High mem (e) SAM - Low mem (f) SAM - High mem

Figure 3: (Left two) Comparison of the loss surfaces for models trained with low (a) and high (b) memorization samples using SGD. The low memorization data points trigger a flatter minimum. (Right four) Loss flatness for SGD and SAM over low and high memorization scores. The larger gap for high memorization can be used to define an approximation of NN memorization as the discrepancy between the minima obtained with SAM and SGD.



(a) Low memorization (b) High memorization

Figure 4: Hessian eigenvalues density distribution for SGD and SAM over low (a) and high (b) memorization scores. The gap between SAM and SGD is greater for high memorization.

eigenvalues distribution of the hessian matrix—a widely adopted metric for measuring the curvature of the loss landscape [17, 43, 52]. Because batch normalization can obfuscate the hessian interpretation, we compute these values on models without batch norm. As lower eigenvalues identify flatter minima, Figure 4 confirms the greater difference between SGD and SAM models on high memorization samples. These novel findings support our hypothesis on the nature of loss flatness for SGD and SAM around memorized samples, hinting at how the discrepancy between SAM and SGD can be used to approximate effectively the NN memorization process.

4.2 SAMIS

Motivated by the above findings, we propose SAMIS, a novel approach that utilizes performance differentials between SAM and SGD to approximate NN memorization. Specifically, SAMIS uses the difference in the prediction probabilities of SAM and SGD for a data point as a proxy for its memorization score. One caveat for this approach is that it cannot be included in the training set because both optimizers for sufficiently large model would fit all training data points to almost-zero-loss, inducing no difference. Kim et al. [29] introduced an entropy-based metric leveraging influence functions to assess the reliance of a model's prediction for a given test point on a limited subset of training data. Their findings suggest that predictions for test points that are highly dependent on specific training examples exhibit a more pronounced discrepancy in test accuracy between models trained with SAM and those trained with SGD. We hypothesize that this phenomenon can be extrapolated to training data points with high memorization scores. Such points, akin to outliers, would similarly exhibit a heavy dependency on a few training examples *were they part of a test set*. In contrast, typical data points, surrounded by numerous similar examples, would likely have a lower dependency, were they in the test set, since its prediction would be influenced by a broader array of training data. Building on this understanding and the intuitions offered in 4.1, we introduce two variants of SAMIS: SAMIS-L and SAMIS-P.

SAMIS-L. We consider the absolute difference in the loss between the models obtained using SAM and SGD, and define SAMIS-L as $S_L(\mathbf{x}_i) = |\mathcal{L}\left(h^{\alpha}(\mathbf{x}_i), y_i\right) - \mathcal{L}\left(h^{\sigma}(\mathbf{x}_i), y_i\right)|$, where h^{α} represents the model optimised using SAM ($h^{SAM}_{\mathcal{S}^{\setminus i}}$), h^{σ} its SGD counterpart ($h^{SGD}_{\mathcal{S}^{\setminus i}}$) and C the number of classes in the dataset.

This simple formulation can be even more resource-intensive than the original memorization formulation, as SAM requires two forward and backward passes. Our method, however, allows us to more reasonably replace $h^{SAM}_{\mathcal{S}^{\setminus i}}$ with $h^{SAM}_{\mathcal{S}^{\setminus\{i,j,k,\ldots,z\}}}$ and $h^{SGD}_{\mathcal{S}^{\setminus i}}$ with $h^{SGD}_{\mathcal{S}^{\setminus\{i,j,k,\ldots,z\}}}$, where $\{i, j, k, ..., z\}$ are data indices removed from the training set and used as test set. Here, the trained models can be reused for computing the scores for all the data points in the test set, namely $\mathbf{x}_i, \mathbf{x}_j, \ldots, \mathbf{x}_z$. While there can be difference in $h^{SAM}_{\mathcal{S}^{\setminus i}}$ against $h^{SAM}_{\mathcal{S}^{\setminus\{i,j,k,\ldots,z\}}}$ and $h^{SGD}_{\mathcal{S}^{\setminus i}}$ against $h^{SGD}_{\mathcal{S}^{\setminus\{i,j,k,\ldots,z\}}}$ separately, we are interested in taking the difference between the two models. Because both models have the same training data and same unseen examples, we can leverage the difference in the loss for each of those examples. This is not the case for memorization formulation in Equation (2) because $h^{\mathcal{A}}_{\mathcal{T}}$ is always trained on the full training dataset. Thus, it is very challenging to generate high quality memorization scores using the setup of Feldman and Zhang [13] while significantly reducing number of models required to be trained – i.e. reducing from 4000 to 100 models in our case. Still, the size of the test subset used to evaluate our metric should be small enough so that typical samples are not overly removed from the training set, potentially becoming atypical samples. Therefore, we can redefine SAMIS-L as:

$$S_L(\mathbf{x}_i) = |\mathcal{L}\left(h^{SAM}_{\mathcal{S}^{\setminus i,\ldots,z}}(\mathbf{x}_i), y_i\right) - \mathcal{L}\left(h^{SGD}_{\mathcal{S}^{\setminus i,\ldots,z}}(\mathbf{x}_i), y_i\right)|. \quad (3)$$

We report the pseudocode for our algorithm in Algorithm 1.

SAMIS-P. We define the SAMIS-P formulation as the average absolute difference in output distribution between the models obtained using SAM and SGD:

$$S_P(\mathbf{x}_i) = \frac{\sum_{c=1}^{C} |P\left(h^{SAM}_{\mathcal{S}^{\setminus i,\ldots,z}}(\mathbf{x}_i) = c\right) - P\left(h^{SGD}_{\mathcal{S}^{\setminus i,\ldots,z}}(\mathbf{x}_i) = c\right)|}{C}. \quad (4)$$

One key difference between the two variants is that SAMIS-P does not require knowledge of the true label, which can be desirable under certain scenarios.

Algorithm 1 shows the pseudo-code to compute SAMIS-L for a single random seed. The algorithm is given number of classes, number of validation points per class, and the training dataset. First, the full training dataset is split into a collection of training and validation sets so that all the data points are included in the validation set for any one split. For each split, SGD and SAM models are trained separately using the split's training set. The absolute loss differential between SGD and SAM models are computed for data points in the validation set. When all splits are done, all data points have a score, and is further min-max normalized. SAMIS-P follows the same procedure except the average difference of the output probabilities vector is used instead of the loss.

Algorithm 1 SAMIS-L

```
function ProxyMemScore(c, p, tr)
    Initialize tr_idx ← {1 to len(tr)}
    Initialize numSplits ← len(tr)/(c * p)
    Initialize lossDiff ← arr of size len(tr)
    trSets, valSets ← splitData(p)
    for split ← 1 to numSplits do
        model_SGD ← train(trSets[split], SGD)
        model_SAM ← train(trSets[split], SAM)
        valSet ← valSets[split]
        for i in valSet do
            dataPoint ← tr[i]
            l_SGD ← loss(model_SGD, dataPoint)
            l_SAM ← loss(model_SAM, dataPoint)
            lossDiff[i] ← |l_SAM − l_SGD|
        end for
    end for
    lossDiff ← minMaxNormalize(lossDiff)
    return lossDiff
end function
function splitData(p)
    for i ← 1 to numSplits do
        valSets[i] = Random sample p indices from each class
        trSets[i] = tr_idx \ valSets[i]
        Exclude valSets[i] from being sampled again
        return trSets, valSets
    end for
end function
```

Complexity. For a dataset with k splits for test subsets and n runs for different number of random seeds, SAMIS requires running nk models each using SAM and SGD. For CIFAR100, we heuristically choose 10% of the data to be used as test subsets, resulting in a total of 10 splits. Using 5 different random seeds, we train a total of $2 \cdot 10 \cdot 5 = 100$ models. This gives us $\approx 40\times$ speed up compared to training 4000 models as done by Feldman and Zhang [13]. We also observe that using fewer training epochs can still achieve a reasonably high correlation, posing a reasonable trade-off between quality and computational cost. We know that low memorization samples are learned in earlier epochs (see Section 3.4), while high memorization samples are learned later. Motivated by this insight, we explore if SAMIS-based compression can achieve higher speed-ups using premature models—i.e. models that are not trained to completion to get zero training loss on each data point. As noted before, if all the training data points are fit to almost-zero-loss, it is challenging to extract useful information from the difference in the loss or output probabilities of the two models. Using premature models can alleviate this problem. To this end, we consider training two models (one using SGD and one using SAM) for a small number of epochs (up to 50 in our experiments) and compute the S_L and S_P proxies directly on the full training set. Figure 5 shows the results obtained when computing SAMIS over a small number of epochs with varying numbers of seeds. We observe that training only two models with SAM and SGD for 30 epochs each can already provide proxies with a good correlation with memorization scores ($\rho \approx 0.7$), bringing down the computation cost further. The total computational cost here is equivalent to only $\approx 40\%$ of the time to fully train one model. Similarly, averaging over multiple random seeds achieves further improvement in quality, reaching $\rho \approx 0.80$ when averaging over 5 seeds. This setup would require training for 300 epochs total – approximately equal to training only a single large NN model –, resulting in a speed-up factor of $\approx 2000\times$.

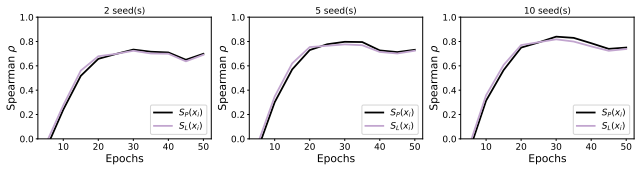


Figure 5: Spearman correlation between memorization and SAMIS on premature training for varying number of epochs and seeds. Training only on 10 seeds for 30 epochs (600 epochs total) SAMIS achieve $\rho \approx 0.83$ proving its efficiency.

Since our focus is on the quality of chosen data samples that can be used for further downstream tasks, we use the original formulation because it achieves a stronger correlation with memorization scores. Lastly, since we are approximating memorization scores for atypicality which is a property of the data, we note that these proxy scores computed using a specific model – like ResNet18 or ResNet50 in our experiments – can be applied to other model architectures and downstream tasks with no further computational overhead. More details about these findings are made available in Appendix E.

4.3 Other Potential Proxies

In this section, we briefly introduce a few different memorization proxies as possible baselines to measure the quality of our SAMIS approximation. Some of these proxies represent previously well-known properties of NNs, while others are newly defined by us.

Sample Forgettability: $F(\mathbf{x}_i)$. We consider example forgetting events as firstly defined by Toneva et al. [49]. Similar to its original formulation, we consider a forgetting event for a single sample $\mathbf{x}_i$ when it is misclassified during the training process at step $t+1$ after having been correctly classified at step t. We extend this definition for a forgetting *event* to introduce the more general concept of sample forgettability. For this metric, we count the forgetting events

that occurred for the same data point while training for T epochs and normalize the obtained value by T. Denoting acc_i^t as $\mathbb{1}_{\hat{y}_i^t = y_i}$, the sample forgettability is defined as:

$$F(\mathbf{x}_i) = \frac{\sum_{t=1}^{T} \mathbb{1}_{acc_i^t > acc_i^{t-1}}}{T}. \tag{5}$$

Epochs to Learn: E2L($\mathbf{x}_i$). Inspired by Toneva et al. [49] which focuses mostly on forgetting events, we devise a new metric E2L. We consider the number of epochs required by the model to learn a sample $\mathbf{x}_i$ and never forget it. It is reasonable to assume that *clean* samples require fewer epochs to be correctly classified, while more atypical samples require a larger amount of epochs, as they go through several forgetting events. The proposed metric ranges from 0 – when a sample $\mathbf{x}_i$ is correctly classified at the very first epoch and never forgotten – to 1—when a sample is never learned. We here consider the *epochs to learn* metric defined as:

$$E2L(\mathbf{x}_i) = \begin{cases} \dfrac{\underset{t}{\text{argmax}} \left\{ acc_i^t > acc_i^{t-1} \right\}}{T} & \text{if } acc_i^T = 1 \\ 1 & \text{otherwise.} \end{cases} \tag{6}$$

Loss curvature: $\gamma(\mathbf{x}_i)$. Garg and Roy [16] analyse the second order of the loss function of trained and partially trained models over dataset samples, highlighting an empirical correlation between the loss curvature and the samples' cleanliness. Inspired by their work, we here follow their approach and leverage the loss curvature of trained models as a possible proxy for the memorization scores. Given a trained models h and a loss function $\mathcal{L}$, Garg and Roy [16] define the loss curvature metric around a sample $\mathbf{x}_i$ as

$$\gamma(\mathbf{x}_i) = \left\| \nabla_{\mathbf{x}_i} \mathcal{L}\left(\mathbf{x}_i + h \cdot z\right) - \nabla_{\mathbf{x}_i} \mathcal{L}(\mathbf{x}_i) \right\|, \tag{7}$$

where $z = \dfrac{sign\left(\nabla_{\mathbf{x}_i} \mathcal{L}(\mathbf{x}_i)\right)}{\left\| sign\left(\nabla_{\mathbf{x}_i} \mathcal{L}(\mathbf{x}_i)\right) \right\|}$ and h represents a hyperparameter empirically set to $h = 3$.

SAM's epsilon: $\epsilon(\mathbf{x}_i)$. Inspired by the empirical findings in [16] on the correlation between sample cleanliness and loss curvature, we consider another alternative proxy for memorization scores that relies on the loss flatness metric as defined by Foret et al. [14]. Specifically, we consider SAM's ϵ, which specifies the perturbation of learnt parameters required to maximally increase the loss within the vicinity of parameters. We compute the norm of SAM's ϵ for each sample in the training set at the end of full model training. Formally, the ϵ proxy for memorization scores is defined as:

$$\epsilon(\mathbf{x}_i) = \left\| \rho \cdot sign\left(\nabla_{\mathbf{x}_i} \mathcal{L}(\mathbf{x}_i)\right) \cdot \frac{|\nabla_{\mathbf{x}_i} \mathcal{L}(\mathbf{x}_i)|}{\sqrt{\left\| \nabla_{\mathbf{x}_i} \mathcal{L}(\mathbf{x}_i) \right\|}} \right\|, \tag{8}$$

where ρ represents the neighborhood size and is set to $\rho = 0.5$ following the experimental findings of [14].

4.4 Proxies Quality Analysis

To analyse the quality of the considered proxies, we compare them against the memorization scores made available by Feldman and Zhang [13] for the CIFAR100 dataset [30]. We compute each proxy measure for each sample $\mathbf{x}_i$ belonging to the training set of the CIFAR100 dataset over $n = 10$ different runs and consider the average score as the final metric. For each run, we vary the initial training seed conditions but keep the learning hyperparameters untouched. To be consistent with the memorization setup we use the same model architecture and the same hyperparameters of [13]. More details regarding the experiment setup can be found in Appendix D. For unbounded proxy values, such as loss curvature, we normalize the scores using min-max normalization to fit them in $[0, 1]$.

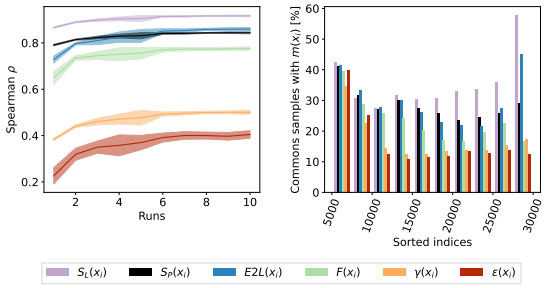


Figure 6: Correlation between memorization and proxies averaged over the number of runs (left) and common samples between memorization and each proxy over low to high scores (right). *SAMIS* and *E2L* are the best proxies.

Proxies vs. Memorization correlation. We compare the rank correlation between each proxy and memorization by using the Spearman rank correlation ρ. Figure 6 shows the correlation between memorization and the proxies averaged over a different number of runs n. These results highlight the promising nature of all the proxies considered, showing how it is possible to reproduce the sample ranking obtained through memorization. The SAMIS metrics defined in Equation (3) achieve the highest correlation reaching up to $\rho = 0.92$. Interestingly, very simple proxies such as *sample forgettability* and *E2L* achieve high correlation with memorization, even if defined in an agnostic manner. We also explore which values of the memorization spectrum each proxy is capable of approximating well. To this end, we count the number of common samples between memorization and each proxy over different splits of the CIFAR100 dataset. We split the dataset into 10 bins each containing 5000 samples sorted according to memorization and proxy scores. For each bin, we compute the percentage of common samples between memorization and the proxy. We report the results in Figure 6, showing how while most proxies can easily approximate low memorization scores, only SAMIS and *E2L* can effectively approximate the high-end of the memorization spectrum. Lastly, we note that when other non-flatness aware algorithms such as Adam and RMSProp are used instead of SGD for SAMIS, comparable correlation is achieved ($\rho = 0.798$, $\rho = 0.75$ for single seed), further corroborating the use of loss geometry for memorization approximation.

Sample visualization. To further investigate the discrepancy between memorization and proxies at the high end of the memorization spectrum, we visualize five samples achieving the lowest and highest scores for the *fox* class of CIFAR100 in Figure 7. Samples with low scores are clean, typical images of orange foxes on a natural background, containing either a close-up of the fox's head or

(a) Low memorization/proxy **(b) High memorization/proxy**

Figure 7: Sample visualization for the *fox* class of CIFAR100 over low (a) and high (b) memorization/proxy scores. Samples with low scores are clean, typical images, while high-scoring examples represent unconventional and atypical images.

the full body of the fox. Several images are matching between memorization and the proxies—especially the SAMIS variants. These results also highlight how all proxies can be used to identify clean, typical samples independently of their capability to approximate the memorization values. On the other hand, the samples extracted from the metrics' highest scores differ quite a bit from one another, while seemingly representing complex, unconventional and atypical samples. More visual examples are available in Appendix A.

5 Proxies for Data Efficient Learning: Experiments and Results

In this section, we leverage the memorization proxies for data summarization, using the proxy estimate to select the data with the lowest/highest values to make up our subsets.

Experimental Setup. We run experiments on CIFAR10, CIFAR100 [30], and SVHN [37] datasets. The CIFAR10 and CIFAR100 datasets contain 50000 training images split into 10 and 100 classes respectively, where the number of samples per class is perfectly balanced. The SVHN dataset contains 73257 training instances of natural images containing digits split into 10 unbalanced classes. All samples are 32-by-32 RGB images. Similarly to Section 3.3, we consider both a data pruning and a few-shot learning setup and compare each proxy-based subset selection's performance against 7 well-known baselines. The selected baselines are comprehensive as they contain the best-performing state-of-the-art approach – namely GraNd [38] accordingly to Guo et al. [18] –, and also the latest publicly available approaches [16, 40]. Accordingly, the selected baselines contain a variety of mechanisms relying on different approaches like gradient matching based methods – such as CRAIG [36] –, bilevel optimization methods – such as Glister [26] –, submodularity based methods – such as GraphCut [21] –, loss based methods – such as GraNd [38], Forgetting [49], InfoBatch [40], and SLo-Curve [16] –, and uniform random sampling. *Forgetting* and *SLo-Curves* correspond to the *sample forgettability* and *loss curvature* proxies respectively and as such we consider them both as baselines and proxies. More details on the experimental setup are available in Appendix D.

5.1 Data Pruning

Figure 8 shows the results for the data pruning setup—i.e., large values of $\mathcal{P}$. The CRAIG [36], Glister [26], and InfoBatch [40] baselines are not shown in the plots, as their performance is not competitive with other state-of-the-art approaches such as GraphCut and GraNd. This is mainly because they are defined for dynamic coreset construction, while here we consider static data summarization.

Across all the selected datasets, the SAMIS proxies represent the best-performing approaches over the majority of $\mathcal{P}$ values. Up to $\mathcal{P} = 0.5$, the S_P-aware data summarization approach outperforms all baselines and most other memorization proxies. Moreover, for some setups, SAMIS-P and SAMIS-L achieve substantial performance improvements over the state-of-the-art, such as for the SVHN dataset, where it is possible to remove up to 70% of the data samples while retaining a test accuracy identical to the full training setup. For the CIFAR10 dataset, it is possible to leverage any of the S_L, S_P, and $E2L$ proxies to prune 50% of the dataset, while keeping the accuracy degradation to be less than 1%.

For smaller values of $\mathcal{P}$, memorization proxies seem to result in higher performance degradation, especially on the CIFAR100 dataset. We hypothesize that such performance drop is due to the increased focus on highly atypical samples which – without the stability of clean typical samples – does not guarantee good training convergence. In these setups, random sampling and GraphCut represent surprisingly strong baselines, as already noted by [18].

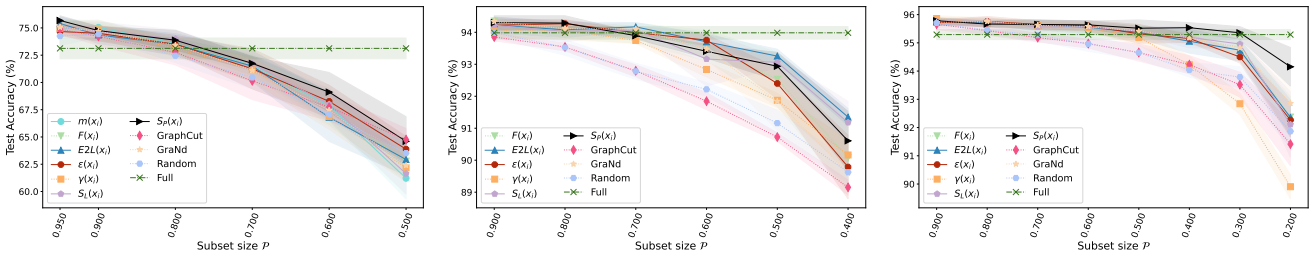
Finally, the results obtained for the CIFAR100 dataset also highlight how S_L performs almost identically to memorization, corroborating the findings of Section 4.4.

5.2 Few-shot Data Summarization

Figure 9 shows the results for the few-shot data summarization. Here, we select $\mathcal{P}$ so that the sampled training subset contains only a handful of images for each class in the dataset. Similarly to Section 5.1, CRAIG, Glister, and InfoBatch performance is not competitive with other state-of-the-art approaches, thus they are not shown.

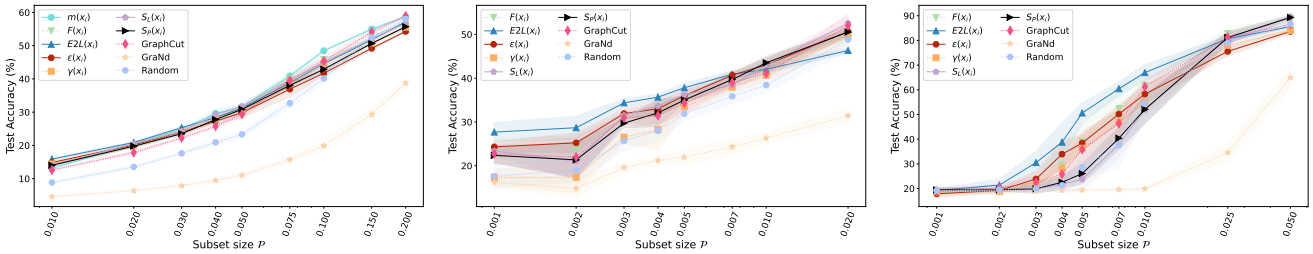
Memorization and its proxies represent the best-performing approaches across all of the considered datasets. For the CIFAR100 dataset, the memorization-aware few-shot data summarization achieves the highest accuracy across all values of $\mathcal{P}$ except 0.01, where $E2L$ works best. Interestingly, S_L and S_P perform close to memorization, up until $\mathcal{P} = 0.075$, where the discrepancy between memorization and its proxies begin to affect the achieved performance. For larger subset sizes – e.g., $\mathcal{P} \in [0.15, 0.2]$ – the number of training images per class becomes less prohibitive, resulting in the state-of-the-art approaches being more effective than memorization proxies. However, for large subsets memorization-aware data pruning still represents the best approach, showing its robustness.

Regarding the CIFAR10 and SVHN datasets, for very small values of $\mathcal{P}$ – e.g., $\mathcal{P} \in [0.001, 0.0075]$ for CIFAR10 and $\mathcal{P} \in [0.001, 0.01]$ for SVHN –, the $E2L$ proxy achieves the highest performance, overcoming the state-of-the-art by a large margin. On the other hand, the S_L and S_P achieve the highest accuracy for increasing values of $\mathcal{P}$—e.g., $\mathcal{P} \in [0.01, 0.02]$ for CIFAR10 and $\mathcal{P} \in [0.025, 0.05]$ for SVHN. These behaviours are due to the nature of the proxy values. The lowest scoring examples in the SAMIS proxies get scores exactly equal to zero, thus generating quite a few ties between data samples, where the sorting procedure is not ideal. Meanwhile, for other proxies like $E2L$, the lowest-scoring examples are more



(a) CIFAR100 (b) CIFAR10 (c) SVHN

Figure 8: Proxies-aware data pruning against state-of-the-art. Memorization/proxies outperform the baselines over most $\mathcal{P}$.



(a) CIFAR100 (b) CIFAR10 (c) SVHN

Figure 9: Proxies-aware few-shot data summarization against state-of-the-art. Memorization/proxies reach the highest accuracy.

spread out, avoiding ties. This hypothesis is confirmed by the sharp performance rise of S_L and S_P for $\mathcal{P} > 0.05$, which is the threshold where ties begin to disappear as the proxy scores become greater than zero. This behaviour represents an opportunity for future analysis, as it highlights how the mixture of various proxies can be used to boost the data summarization performance.

Overall, these results prove that it is possible to approximate the expensive memorization scores using our proxies and retain even higher performance on both data summarization tasks, confirming the hypothesis made in Section 3.4 about the correlation between data-efficient learning and the NTK velocity. Indeed, the contribution of proxy scores to the NTK gram matrix velocity shows similar behaviour to the one shown in Figure 2 for memorization (see Appendix C).

6 Conclusion

In this paper, we investigate the effectiveness of leveraging memorization scores and their proxies for dataset summarization. We first show that a sampling of data attaining the lowest (highest) memorization scores results in more effective learning when a small (large) number of training examples is considered. To account for the resource hungriness of memorization computation we propose SAMIS, a proxy metric leveraging the discrepancy between the minimum obtained when training with SGD and SAM. The proposed proxy relies on the finding that clean typical samples trigger flatter minima, while atypical examples reach sharper minima. We compare the SAMIS proxies against several metrics, showcasing their strong correlation with memorization scores and their capability of generalizing across architectures. Thereafter, we replicate the memorization-based dataset summarization leveraging the proposed proxies and compare them with state-of-the-art approaches, highlighting their effectiveness.

Acknowledgements

This paper was partially supported by *(i)* Central Indiana Corporate Partnership AnalytiXIN Initiative; and *(ii)* the "ENGINES — ENGineering INtElligent Systems around intelligent agent technologies" project funded by the Italian MUR program "PRIN 2022" under grant number 20229ZXBZM.

References

[1] Georges Aad et al. 2012. Observation of a new particle in the search for the Standard Model Higgs boson with the ATLAS detector at the LHC. *Phys. Lett. B* 716 (2012), 1–29. arXiv:1207.7214

[2] Andrea Agiollo, Enkeleda Bardhi, Mauro Conti, Riccardo Lazzeretti, Eleonora Losiouk, and Andrea Omicini. 2023. GNN4IFA: Interest Flooding Attack Detection With Graph Neural Networks. In *8th IEEE European Symposium on Security and Privacy, EuroS&P, Delft, Netherlands, July 3-7, 2023*. IEEE, 615–630.

[3] Marouane Birjali, Mohammed Kasri, and Abderrahim Beni Hssane. 2021. A Comprehensive Survey on Sentiment Analysis: Approaches, Challenges and Trends. *Knowledge Based Systems* 226 (2021), 107134.

[4] George Cazenavette, Tongzhou Wang, Antonio Torralba, Alexei A. Efros, and Jun-Yan Zhu. 2022. Dataset Distillation by Matching Training Trajectories. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops, CVPR Workshops, New Orleans, LA, USA, June 19-20, 2022*. IEEE, 4749–4758.

[5] Yutian Chen, Max Welling, and Alexander J. Smola. 2010. Super-Samples from Kernel Herding. In *26th Conference on Uncertainty in Artificial Intelligence, UAI, Catalina Island, CA, USA, July 8-11, 2010*. AUAI Press, 109–116.

[6] Samuel Fuller Dodge and Lina J. Karam. 2016. Understanding How Image Quality Affects Deep Neural Networks. In *8th International Conference on Quality of Multimedia Experience, QoMEX, Lisbon, Portugal, June 6-8, 2016*. IEEE, 1–6.

[7] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2021. An Image is

Worth 16x16 Words: Transformers for Image Recognition at Scale. In *9th International Conference on Learning Representations, ICLR, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net.

[8] Jiawei Du, Yidi Jiang, Vincent Y. F. Tan, Joey Tianyi Zhou, and Haizhou Li. 2023. Minimizing the Accumulated Trajectory Error to Improve Dataset Distillation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR, Vancouver, BC, Canada, June 17-24, 2023*. IEEE, 3749–3758.

[9] Simon S. Du, Yining Wang, Xiyu Zhai, Sivaraman Balakrishnan, Ruslan Salakhutdinov, and Aarti Singh. 2018. How Many Samples are Needed to Estimate a Convolutional Neural Network?. In *30th Annual Conference on Neural Information Processing Systems, NeurIPS, December 3-8, 2018, Montréal, Canada*. 371–381.

[10] Melanie Ducoffe and Frédéric Precioso. 2018. Adversarial Active Learning for Deep Networks: a Margin Based Approach. *CoRR* abs/1802.09841 (2018). arXiv:1802.09841

[11] Dumitru Erhan, Christian Szegedy, Alexander Toshev, and Dragomir Anguelov. 2014. Scalable Object Detection Using Deep Neural Networks. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR , Columbus, OH, USA, June 23-28, 2014*. IEEE Computer Society, 2155–2162.

[12] Vitaly Feldman. 2020. Does Learning Require Memorization? A Short Tale about a Long Tail. In *52nd Annual ACM SIGACT Symposium on Theory of Computing, STOC, Chicago, IL, USA, June 22-26, 2020*. ACM, 954–959.

[13] Vitaly Feldman and Chiyuan Zhang. 2020. What Neural Networks Memorize and Why: Discovering the Long Tail via Influence Estimation. In *34th Annual Conference on Neural Information Processing Systems, NeurIPS, December 6-12, 2020, virtual*.

[14] Pierre Foret, Ariel Kleiner, Hossein Mobahi, and Behnam Neyshabur. 2021. Sharpness-aware Minimization for Efficiently Improving Generalization. In *9th International Conference on Learning Representations, ICLR, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net.

[15] Stanislav Fort, Gintare Karolina Dziugaite, Mansheej Paul, Sepideh Kharaghani, Daniel M. Roy, and Surya Ganguli. 2020. Deep learning Versus Kernel Learning: an Empirical Study of Loss Landscape Geometry and the Time Evolution of the Neural Tangent Kernel. In *32nd Annual Conference on Neural Information Processing Systems, NeurIPS, December 6-12, 2020, virtual*.

[16] Isha Garg and Kaushik Roy. 2023. Samples with Low Loss Curvature Improve Data Efficiency. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR, Vancouver, BC, Canada, June 17-24, 2023*. IEEE.

[17] Behrooz Ghorbani, Shankar Krishnan, and Ying Xiao. 2019. An Investigation into Neural Net Optimization via Hessian Eigenvalue Density. In *36th International Conference on Machine Learning, ICML, 9-15 June 2019, Long Beach, California, USA (Proceedings of Machine Learning Research, Vol. 97)*. PMLR, 2232–2241.

[18] Chengcheng Guo, Bo Zhao, and Yanbing Bai. 2022. DeepCore: A Comprehensive Library for Coreset Selection in Deep Learning. In *33rd International Conference on Database and Expert Systems Applications, DEXA, Vienna, Austria, August 22-24, 2022 (Lecture Notes in Computer Science, Vol. 13426)*. Springer, 181–195.

[19] Sariel Har-Peled and Soham Mazumdar. 2004. On Coresets for K-means and K-median Clustering. In *36th Annual ACM Symposium on Theory of Computing, Chicago, IL, USA, June 13-16, 2004*. ACM, 291–300.

[20] Rishabh K. Iyer and Jeff A. Bilmes. 2013. Submodular Optimization with Submodular Cover and Submodular Knapsack Constraints. In *27th Annual Conference on Neural Information Processing Systems, NeurIPS, December 5-8, 2013, Lake Tahoe, Nevada, United States*. 2436–2444.

[21] Rishabh K. Iyer, Ninad Khargoankar, Jeff A. Bilmes, and Himanshu Asanani. 2021. Submodular Combinatorial Information Measures with Applications in Machine Learning. In *Algorithmic Learning Theory Conference, Virtual 16-19 March 2021 (Proceedings of Machine Learning Research, Vol. 132)*. PMLR, 722–754.

[22] Pavel Izmailov, Dmitrii Podoprikhin, Timur Garipov, Dmitry P. Vetrov, and Andrew Gordon Wilson. 2018. Averaging Weights Leads to Wider Optima and Better Generalization. In *34th Conference on Uncertainty in Artificial Intelligence, UAI, Monterey, California, USA, August 6-10, 2018*. AUAI Press, 876–885.

[23] Tyler B. Johnson and Carlos Guestrin. 2018. Training Deep Models Faster with Robust, Approximate Importance Sampling. In *30th Annual Conference on Neural Information Processing Systems, NeurIPS, December 3-8, 2018, Montréal, Canada*.

[24] Angelos Katharopoulos and François Fleuret. 2018. Not All Samples Are Created Equal: Deep Learning with Importance Sampling. In *35th International Conference on Machine Learning, ICML, Stockholmsmässan, Stockholm, Sweden, July 10-15, 2018 (Proceedings of Machine Learning Research, Vol. 80)*. PMLR, 2530–2539.

[25] KrishnaTeja Killamsetty, Durga Sivasubramanian, Ganesh Ramakrishnan, Abir De, and Rishabh K. Iyer. 2021. GRAD-MATCH: Gradient Matching based Data Subset Selection for Efficient Deep Model Training. In *38th International Conference on Machine Learning, ICML, 18-24 July 2021, Virtual Event (Proceedings of Machine Learning Research, Vol. 139)*. PMLR, 5464–5474.

[26] KrishnaTeja Killamsetty, Durga Sivasubramanian, Ganesh Ramakrishnan, and Rishabh K. Iyer. 2021. GLISTER: Generalization based Data Subset Selection for Efficient and Robust Learning. In *35th AAAI Conference on Artificial Intelligence, AAAI, Virtual Event, February 2-9, 2021*. AAAI Press, 8110–8118.

[27] KrishnaTeja Killamsetty, Xujiang Zhao, Feng Chen, and Rishabh K. Iyer. 2021. RETRIEVE: Coreset Selection for Efficient and Robust Semi-Supervised Learning. In *35th Annual Conference on Neural Information Processing Systems 2021, NeurIPS, December 6-14, 2021, virtual*. 14488–14501.

[28] Jang-Hyun Kim, Jinuk Kim, Seong Joon Oh, Sangdoo Yun, Hwanjun Song, Joonhyun Jeong, Jung-Woo Ha, and Hyun Oh Song. 2022. Dataset Condensation via Efficient Synthetic-Data Parameterization. In *International Conference on Machine Learning, ICML, 17-23 July 2022, Baltimore, Maryland, USA (Proceedings of Machine Learning Research, Vol. 162)*. PMLR, 11102–11118.

[29] Young In Kim, Pratiksha Agrawal, Johannes O. Royset, and Rajiv Khanna. 2024. On Memorization and Privacy Risks of Sharpness Aware Minimization. arXiv:2310.00488

[30] Alex Krizhevsky, Geoffrey Hinton, et al. 2009. Learning Multiple Layers of Features from Tiny Images. (2009).

[31] Shiye Lei and Dacheng Tao. 2024. A Comprehensive Survey of Dataset Distillation. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46, 1 (2024), 17–32.

[32] Hao Li, Zheng Xu, Gavin Taylor, Christoph Studer, and Tom Goldstein. 2018. Visualizing the Loss Landscape of Neural Nets. In *32nd Annual Conference on Neural Information Processing Systems, NeurIPS 2018*, Vol. 31. Curran Associates, Inc.

[33] Yahui Liu, Enver Sangineto, Wei Bi, Nicu Sebe, Bruno Lepri, and Marco De Nadai. 2021. Efficient Training of Visual Transformers with Small Datasets. In *Annual Conference on Neural Information Processing Systems, NeurIPS, December 6-14, 2021, virtual*. 23818–23830.

[34] Katerina Margatina, Giorgos Vernikos, Loïc Barrault, and Nikolaos Aletras. 2021. Active Learning by Acquiring Contrastive Examples. In *Conference on Empirical Methods in Natural Language Processing, EMNLP, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021*. Association for Computational Linguistics.

[35] Folker Meyer, Saurabh Bagchi, Somali Chaterji, Wolfgang Gerlach, Ananth Grama, Travis Harrison, Tobias Paczian, William L. Trimble, and Andreas Wilke. 2019. MG-RAST version 4 - lessons learned from a decade of low-budget ultra-high-throughput metagenome analysis. *Briefings in Bioinformatics* 20, 4 (2019).

[36] Baharan Mirzasoleiman, Jeff A. Bilmes, and Jure Leskovec. 2020. Coresets for Data-efficient Training of Machine Learning Models. In *37th International Conference on Machine Learning, ICML, 13-18 July 2020, Virtual Event (Proceedings of Machine Learning Research, Vol. 119)*. PMLR, 6950–6960.

[37] Yuval Netzer, Tao Wang, Adam Coates, Alessandro Bissacco, Bo Wu, and Andrew Y Ng. 2011. Reading Digits in Natural Images with Unsupervised Feature Learning. (2011).

[38] Mansheej Paul, Surya Ganguli, and Gintare Karolina Dziugaite. 2021. Deep Learning on a Data Diet: Finding Important Examples Early in Training. In *35th Annual Conference on Neural Information Processing Systems, NeurIPS, December 6-14, 2021, virtual*. 20596–20607.

[39] Marek Pawlicki, Rafal Kozik, and Michal Choras. 2022. A Survey on Neural Networks for (Cyber-)security and (Cyber-)security of Neural Networks. *Neurocomputing* 500 (2022), 1075–1087.

[40] Ziheng Qin, Kai Wang, Zangwei Zheng, Jianyang Gu, Xiangyu Peng, Daquan Zhou, and Yang You. 2023. InfoBatch: Lossless Training Speed Up by Unbiased Dynamic Data Pruning. *arXiv* abs/2303.04947 (2023).

[41] Vikram Ravindra, Petros Drineas, and Ananth Grama. 2021. Constructing Compact Signatures for Individual Fingerprinting of Brain Connectomes. *Frontiers in Neuroscience* 15 (2021).

[42] Pengzhen Ren, Yun Xiao, Xiaojun Chang, Po-Yao Huang, Zhihui Li, Brij B. Gupta, Xiaojiang Chen, and Xin Wang. 2022. A Survey of Deep Active Learning. *Comput. Surveys* 54, 9 (2022), 180:1–180:40.

[43] Adepu Ravi Sankar, Yash Khasbage, Rahul Vigneswaran, and Vineeth N. Balasubramanian. 2021. A Deeper Look at the Hessian Eigenspectrum of Deep Neural Networks and its Applications to Regularization. In *35th AAAI Conference on Artificial Intelligence, AAAI, Virtual Event, February 2-9, 2021*. AAAI Press.

[44] Aliaksei Severyn and Alessandro Moschitti. 2015. Twitter Sentiment Analysis with Deep Convolutional Neural Networks. In *International ACM SIGIR Conference on Research and Development in Information Retrieval, Santiago, Chile, August 9-13, 2015*. ACM, 959–962.

[45] Vipul Sharma and Roohie Naaz Mir. 2020. A Comprehensive and Systematic Look Up into Deep Learning based Object Detection Techniques: A Review. *Computer Science Review* 38 (2020), 100301.

[46] Karen Simonyan and Andrew Zisserman. 2015. Very Deep Convolutional Networks for Large-Scale Image Recognition. In *3rd International Conference on Learning Representations, ICLR, San Diego, CA, USA, May 7-9, 2015*.

[47] Justin S. Smith, Roman Zubatyuk, Benjamin Nebgen, Nicholas Lubbers, Kipton Barros, Adrian E. Roitberg, Olexandr Isayev, and Sergei Tretiak. 2020. The ANI-1ccx and ANI-1x data sets, coupled-cluster and density functional theory properties for molecules. *Scientific Data* 7, 1 (2020).

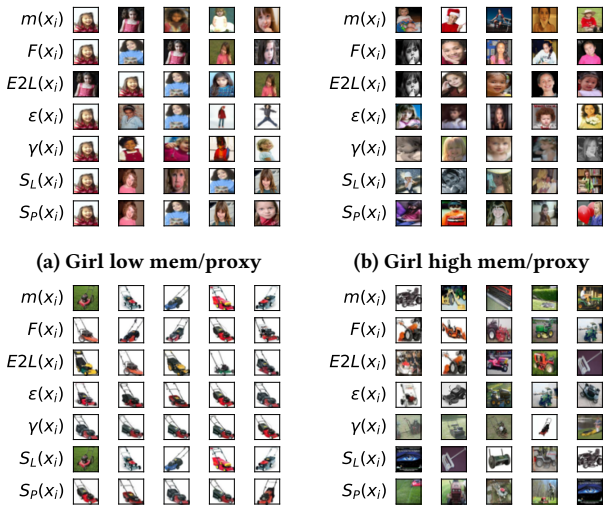
[48] Ben Sorscher, Robert Geirhos, Shashank Shekhar, Surya Ganguli, and Ari Morcos. 2022. Beyond neural scaling laws: beating power law scaling via data pruning. In *Advances in Neural Information Processing Systems*, Vol. 35. Curran Associates, Inc., 19523–19536.

[49] Mariya Toneva, Alessandro Sordoni, Remi Tachet des Combes, Adam Trischler, Yoshua Bengio, and Geoffrey J. Gordon. 2019. An Empirical Study of Example

Forgetting during Deep Neural Network Learning. In *7th International Conference on Learning Representations, ICLR, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net.
[50] Tongzhou Wang, Jun-Yan Zhu, Antonio Torralba, and Alexei A. Efros. 2020. Dataset Distillation. arXiv:1811.10959
[51] Shuo Yang, Zeke Xie, Hanyu Peng, Min Xu, Mingming Sun, and Ping Li. 2023. Dataset Pruning: Reducing Training Data by Examining Generalization Influence. arXiv:2205.09329
[52] Zhewei Yao, Amir Gholami, Kurt Keutzer, and Michael W. Mahoney. 2020. PyHessian: Neural Networks Through the Lens of the Hessian. In *IEEE International Conference on Big Data (IEEE BigData), Atlanta, GA, USA, December 10-13, 2020*. IEEE, 581–590.
[53] Bo Zhao, Konda Reddy Mopuri, and Hakan Bilen. 2021. Dataset Condensation with Gradient Matching. In *9th International Conference on Learning Representations, ICLR, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net.

A Proxies correlation over classes

To better understand the impact of class complexity, we investigate the class-wise correlation between memorization and proxy scores. Once again, the SAMIS and *E2L* proxies represent the most effective metrics, as shown in Figure 11. Interestingly, the correlation behaviour of most proxies is rather turbulent. Only the metrics reaching an overall Pearson correlation greater than 0.8 showcase a sufficiently uniform correlation value over all classes. The *girl* class represents the most complex instance to be well approximated by proxies. Conversely, the *lawn mower* class represents the easiest instance to be well approximated. For each of the peculiar – i.e., complex (Figures 10a and 10b) or easy (Figures 10c and 10d) – classes we plot the lowest and highest scoring samples, similarly to what is done in Figure 7 for the class *fox*.



(a) Girl low mem/proxy **(b) Girl high mem/proxy**

(c) Lawn mower low mem/proxy **(d) Lawn mower high mem/proxy**

Figure 10: Sample visualization for the *girl* ((a) and (b)) and *lawn mower* ((c) and (d)) classes of CIFAR100 over low (left) and high (right) memorization/proxy scores.

B Impact of Memorization on Loss Flatness Across Datasets and Architectures

Here we present the loss landscape visualizations for models trained on low 30% and high 30% data sorted according to SAMIS-L in Figure 12. To verify consistency across different datasets and architectures, we use CIFAR10 with Resnet18 and VGG19. Because we do not have readily available memorization scores for CIFAR10, we use S_L as approximation for the memorization scores since it was the proxy achieving the highest rank correlation score. The results are consistent with the visualization for models using memorization scores. For both architectures, models trained on typical, clean – i.e., low S_L – samples showcase a flatter loss landscape compared to models trained on atypical – i.e., high S_L – samples. These findings corroborate our hypothesis on the correlation between sample typicality and loss flatness, highlighting that clean samples trigger flat minima independently of the selected dataset and NN model.

C NTK Velocity for Proxies

To study if the findings presented in Section 3.4 for the correlation between memorization scores and NTK velocity translate to the memorization proxies, we here consider computing the NTK velocity of the samples sorted by each proxy score. We train a ResNet18 model on the CIFAR10 dataset and compute the velocity over various epochs t. Figure 13 shows the obtained results. As expected, the NTK velocity is proportional to the samples' proxy scores for all proxies. The same velocity behaviour translates from memorization scores to its proxies, without loss of generality.

D Experiments Setup

Memorization Proxies. To mimic as close as possible the setup used by Feldman and Zhang [13] to compute the memorization scores on the CIFAR100 dataset, we use the ResNet50 architecture, and SGD with momentum 0.9, a batch size of 512 and a base learning rate of 0.4. We train each model for 160 training epochs, in which the learning rate is scheduled to grow linearly from 0 to the base learning rate in the first 15% of epochs, and then decay linearly back to 0 in the remaining epochs.

For the CIFAR10 and SVHN datasets, we use a ResNet18 for proxy extraction, since memorization scores are not available for these setups. The other hyperparameters are left untouched except for the number of epochs and base learning rate. The number of training epochs is set to 160 for the CIFAR10 dataset and 80 for SVHN, while the base learning rate is set to 0.1 for both datasets.

Data Pruning. We train a ResNet50 model over the CIFAR100 dataset and a ResNet18 architecture over all the remaining datasets. The training hyperparameters are shown in Table 1.

Few-shot Data Summarization. We train a ResNet18 model over all the selected datasets. The training hyperparameters are shown in Table 1.

E Cross-Architecture Experiments

We here measure to what extent the proposed memorization proxies generalize across architectures different from the one used to compute them. To this end, we train a VGG19 model [46] over the CIFAR10 dataset and a ViT model [7] over the CIFAR100 dataset.

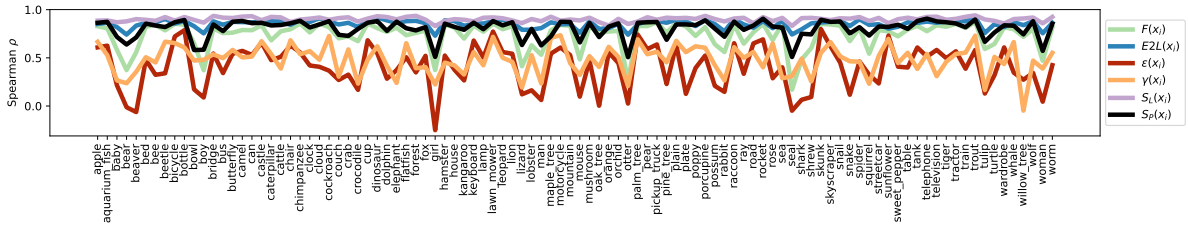
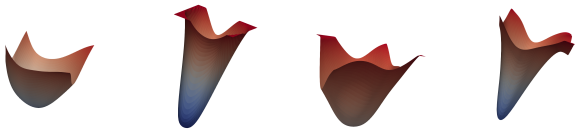


Figure 11: Pearson correlation between memorization and proxies over each CIFAR100 class.



(a) Resnet18 Low mem **(b) Resnet18 High mem** **(c) VGG19 Low mem** **(d) VGG19 High mem**

Figure 12: Comparison of the loss flatness for models trained with low (a) and high (b) SAMIS-L scores for CIFAR10 using Resnet18 and VGG19.

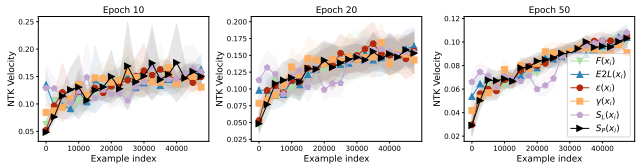
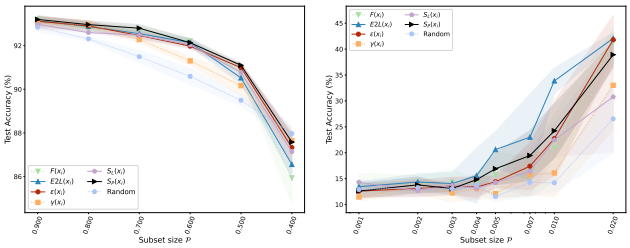


Figure 13: NTK velocity for CIFAR-10 samples sorted in ascending order by proxy scores.

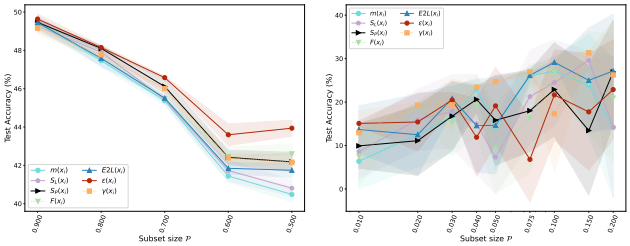
Setup	Dataset	$\mathcal{P}$	Epochs	λ	β	λ_s	λ_f
Data Pruning	CIFAR100	[0.5, 1]	160	0.1	256	65	0.1
	CIFAR10	[0.4, 1]	160	0.1	256	50	0.1
	SVHN	[0.2, 1]	80	0.1	256	25	0.1
Few-shot Summarization	CIFAR100	[0.01, 0.2]	160	0.1	32	65	0.1
	CIFAR10	[0.001, 0.02]	160	0.1	32	65	0.1
	SVHN	[0.001, 0.05]	80	0.1	32	25	0.1

Table 1: Experiments hyperparameters for data pruning and few-shot data summarization. λ = starting learning rate, β = batch size, λ_s = number of epochs after which the learning rate decays, λ_f = learning rate decay factor.

We rely on the same data pruning and few-shot data summarization setups of Section 5. Figures 14a and 14b show the results obtained for the VGG19 model. We compare only against the uniform random sampling approach, as memorization scores are not readily available for CIFAR10 and since random sampling is the only other approach that does not require training any models to select the optimization subset. The obtained results highlight how most proxies can generalize well over unseen architectures. In both setups, the SAMIS and *E2L* proxies represent the best approaches, overcoming the uniform random sampling.



(a) Data pruning **(b) Few-shot data summarization**



(c) Data pruning **(d) Few-shot data summarization**

Figure 14: Proxies-aware data summarization when proxies are applied to the VGG19 model on CIFAR10 ((a) and (b)) or the ViT model on CIFAR100 ((c) and (d)).

Figures 14c and 14d show the results obtained for the ViT model. We compare against the memorization baselines – readily available for CIFAR100 – to measure SAMIS effectiveness against the original definition of memorization. The obtained results highlight how SAMIS can generalize better than memorization over the data pruning setup. Meanwhile, for few-shot summarization, almost all selection criteria are not performing well, given the complexity of training transformer models on very few data [33]. The results show how it is possible to compute the memorization proxy scores only once for each dataset and reuse them across different architectures.